Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2017.DOI

Efficient and Reproducible Biomedical Named Entity Recognition: A Comparative Evaluation of Full Fine-Tuning, LoRA, and QLoRA Using PubMedBERT

CHIGURUPATI VENKAT SAI KIRAN[1], AND DR. SUJATHA M[1]

[1]School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Chennai, Tamil Nadu 600127, India (e-mail: chigurupati.venkat2025@vitstudent.ac.in; sujatha.m@vit.ac.in)

Corresponding author: Chigurupati Venkat Sai Kiran (e-mail: chigurupati.venkat2025@vitstudent.ac.in).

ABSTRACT Biomedical named entity recognition requires models to identify chemically and clinically meaningful spans while remaining practical to train on limited hardware and labeled data. This paper presents a controlled comparison of full fine-tuning, low-rank adaptation, and quantized low-rank adaptation for chemical and disease recognition with PubMedBERT on the BioCreative V Chemical Disease Relation corpus. The study comprises 24 training runs spanning three adaptation methods, four training-set sizes (500, 1,000, 3,000, and all 5,228 training sentences), and two random seeds. Evaluation uses strict entity-level span and type matching. On the full training set, mean test F1 scores are 0.8913 for full fine-tuning, 0.8955 for low-rank adaptation, and 0.8964 for quantized low-rank adaptation. Quantized low-rank adaptation reduces the measured peak device-memory allocation from 2.241 GB to 0.496 GB, while both adapter methods update only 5.312 million parameters compared with 108.895 million for full fine-tuning. The differences become more pronounced with only 500 training sentences. LoRA and QLoRA reach F1 scores of 0.8020 and 0.7853, respectively, whereas full fine-tuning reaches 0.7098. At this smaller training size, the separation between the methods is therefore much larger than the difference observed after using the complete training partition. These results indicate that parameter-efficient adaptation can retain full-data performance while providing substantially different resource and low-data behavior. The study documents the training configuration, random seeds, evaluation protocol, hardware environment, and deployment artifacts to support reproducibility.

INDEX TERMS Biomedical natural language processing, low-rank adaptation, named entity recognition, PubMedBERT, quantization

I. INTRODUCTION

BIOMEDICAL literature contains a large and continually expanding body of information about chemicals, diseases, treatments, adverse effects, and biological mechanisms. Named entity recognition (NER) is an important information-extraction step because it converts unstructured biomedical text into structured entity mentions that can subsequently support relation extraction, search, curation, and knowledge-base construction. Unlike general-domain NER, biomedical NER must cope with long technical expressions, abbreviations, compositional terminology, spelling variation, and entity boundaries that are difficult to infer from surface form alone [1]–[3].

Transformer encoders changed the practical baseline for NER. BERT introduced bidirectional contextual pretraining followed by task-specific fine-tuning [4], while the Transformer architecture established the self-attention mechanism underlying such models [5]. Domain-specific pretraining further improved biomedical language understanding: BioBERT continued pretraining on biomedical corpora [2], SciBERT used scientific text and a scientific vocabulary [6], and PubMedBERT was pretrained from scratch on biomedical text

with a vocabulary designed for that domain [7]. These models have become strong foundations for biomedical NER across multiple corpora [8], [9].

The standard adaptation strategy is full fine-tuning, in which essentially all model parameters are updated for the downstream task. Although effective, this strategy couples task training to the memory requirements of the complete model and optimizer state. This becomes important when biomedical models are trained on commodity GPUs or when multiple task-specific versions must be maintained. Parameter-efficient fine-tuning (PEFT) methods address this issue by freezing most pretrained weights and updating a much smaller parameter subset. Adapters, prefix tuning, prompt tuning, BitFit, and LoRA are representative approaches [10]–[14]. QLoRA extends this idea by storing the frozen backbone in 4-bit NormalFloat representation while learning LoRA adapters, together with double quantization and memory-management techniques [15].

Despite the rapid growth of PEFT research, an important experimental question remains specific to biomedical NER: how do full fine-tuning, LoRA, and QLoRA behave when the backbone, dataset, evaluation protocol, data subsets, and random seeds are held constant? Existing biomedical NER literature contains strong model comparisons and task-specific improvements, but resource measurements and controlled data-size analyses are less consistently reported. Generalization to novel biomedical mentions is also a known concern, meaning that benchmark F1 alone does not fully describe practical reliability [16].

This work therefore studies three adaptation paradigms under one controlled experimental design. The contribution is not a new neural architecture; rather, it is a reproducible empirical characterization of the accuracy–resource–data tradeoff for PubMedBERT on BC5CDR. The study makes four contributions:

1) It compares full fine-tuning, LoRA, and QLoRA on the same PubMedBERT token-classification task and evaluation pipeline.
2) It evaluates four labeled-data regimes (500, 1,000, 3,000, and all 5,228 training sentences), exposing behavior that is hidden by a single full-data score.
3) It reports trainable parameters, peak CUDA memory allocation, and training time alongside entity-level precision, recall, and F1 on a 4 GB-class consumer GPU.
4) It repeats every configuration with seeds 42 and 123 and reports mean and standard deviation, while releasing the implementation and deployment artifacts to support reproduction.

A key methodological distinction is that this study is a controlled comparison rather than a leaderboard exercise. The backbone, label space, train/validation/test partitions, maximum sequence length, optimization schedule, sampled subsets, evaluation library, and random seeds are fixed across methods. This design holds the backbone, dataset partitions, data-size grid, evaluation protocol, and random seeds constant while comparing the specified full fine-tuning, LoRA, and QLoRA adaptation configurations. The study therefore targets a narrower but more reproducible question than a broad comparison of unrelated biomedical NER systems.

II. RELATED WORK

A. BIOMEDICAL LANGUAGE MODELS AND NER

The progression from BERT to biomedical pretrained encoders has produced substantial gains in biomedical text mining. BioBERT demonstrated that continued pretraining on biomedical corpora can improve NER, relation extraction, and question answering [2]. SciBERT showed that scientific-domain pretraining and a domain-specific vocabulary can benefit scientific NLP [6]. ClinicalBERT explored adaptation to clinical notes [17], while PubMedBERT used a biomedical vocabulary and domain-specific pretraining corpus from the beginning [7]. LinkBERT subsequently emphasized document-link structure during pretraining [18], illustrating that the pretraining objective and corpus structure can influence downstream performance.

For biomedical NER specifically, the BC5CDR corpus has been a major benchmark since its introduction for chemical and disease mentions and relations [1]. Subsequent studies have explored sequence labeling, machine reading comprehension, multitask learning, and domain-specific pretrained encoders. Sun et al. reformulated biomedical NER as machine reading comprehension and reported strong results across six datasets [19]. Multi-task and feature-aware approaches have also been proposed to exploit shared structure across biomedical entity types [20]. More recent work has examined instruction-tuned large language models for biomedical NER, but encoder-based domain-specific models remain attractive when the task is naturally formulated as token classification [21], [22].

Reliability and generalization are important complements to benchmark performance. Giorgi and Bader investigated reliable biomedical NER and highlighted issues related to model robustness and evaluation [3]. Kim and Kang showed that models with strong benchmark scores can still struggle with novel biomedical entities and synonyms [16]. These observations motivate controlled experiments that distinguish changes in adaptation method from changes in data, backbone, or evaluation.

B. PARAMETER-EFFICIENT FINE-TUNING

Adapters insert small trainable modules into a frozen pretrained network [10]. Prefix tuning and prompt tuning reduce the number of task-specific parameters by optimizing continuous prompts rather than the entire model [11], [12]. BitFit takes an even smaller route by tuning only bias parameters [13]. LoRA instead represents the weight update as a low-rank product, avoiding direct updates to the original matrix while adding relatively few trainable parameters [14]. AdapterHub and related infrastructure made adapter-based research easier to reproduce and compose [23], [24].

QLoRA combines LoRA with 4-bit quantization of the frozen backbone and introduces NormalFloat4, double quan-

tization, and paged optimizers [15]. Its original motivation was very large generative models, but the underlying memory principle is also relevant to smaller biomedical encoders when the limiting resource is GPU memory. Importantly, QLoRA does not imply that quantization always improves task quality; its principal attraction is reducing the memory footprint of the frozen base model while retaining adapter-based optimization.

C. POSITION OF THIS STUDY

The present work differs from a model leaderboard study. The same PubMedBERT checkpoint, tokenization, maximum sequence length, data subsets, optimizer schedule, stopping criterion, evaluation library, and seeds are used across all three adaptation methods. This design isolates the adaptation strategy as the principal experimental factor. The data-size ablation additionally tests whether the relative behavior of the methods changes when labeled biomedical data are scarce.

The literature matrix is used as context rather than as a score-by-score ranking. The papers cover different datasets, formulations, and aims, so their reported numbers are not treated as directly competing results. Its main purpose is to show what is held constant in this study: one biomedical encoder and benchmark, three adaptation choices, several labeled-data levels, and measurements of the training resources.

D. INTERPRETATION OF THE LITERATURE GAP

The paper makes a deliberately limited novelty claim. It introduces neither a new pretraining objective nor a new LoRA formulation or decoding architecture. Instead, it places full fine-tuning, LoRA, and QLoRA in the same PubMedBERT–BC5CDR experiment and changes the amount of labeled training data while recording GPU cost. This controlled setup is useful because PEFT conclusions can shift with the task definition, tuning choices, validation procedure, and amount of supervision [25], [26].

Reproduction was considered while the experiment was being specified, not only when the manuscript was written. Earlier repeatability work on QLoRA [31] also motivates keeping more than one random initialization in view. The present study uses two seeds, which is only a small check and cannot establish statistical stability. Both runs are therefore retained in the appendix so that the reported means can be traced back to the individual observations.

III. MATERIALS AND METHODS

A. RESEARCH QUESTIONS

The experiment is organized around two questions. First, can LoRA and QLoRA approach or exceed the entity-level F1 of full fine-tuning while updating substantially fewer parameters and consuming less GPU memory? Second, does the relative behavior change when the available labeled training set is reduced from the full corpus to 3,000, 1,000, or 500 sentences?

B. DATASET

The BioCreative V Chemical Disease Relation (BC5CDR) corpus contains biomedical abstracts annotated for chemical and disease mentions and their relations [1]. The implementation uses the Hugging Face representation of BC5CDR. The experimental split contains 5,228 training sentences, 5,330 validation sentences, and 5,865 test sentences. The NER label space is the five-class BIO scheme. The implementation uses the exact numeric mapping $0 : O$, $1 : B$-Chemical, $2 : B$-Disease, $3 : I$-Disease, and $4 : I$-Chemical; the semantic label set is therefore O, B-Chemical, I-Chemical, B-Disease, and I-Disease. The test split contains 5,384 chemical mentions and 4,424 disease mentions in the reported evaluation.

The three official partitions contain 16,423 sentences in total (5,228 train, 5,330 validation, and 5,865 test), consistent with the project architecture. For the data-size experiment, four training sizes are evaluated: 500, 1,000, 3,000, and all 5,228 training sentences. The same sampled subsets are used across methods so that method comparisons do not confound adaptation with different examples. Subset construction is deterministic, while the two training seeds (42 and 123) probe stochastic variation in optimization. Each method–size pair is therefore evaluated twice, producing 24 runs in total. The test split is never downsampled and is not used for checkpoint selection.

C. BACKBONE AND TOKENIZATION

The backbone is PubMedBERT-base-uncased-abstract, using the exact project model identifier recorded in the project artifacts and reproducibility record, and is designed for biomedical text [7]. The token-classification backbone contains 108.895 million parameters and 12 Transformer encoder layers. Each sentence is tokenized using the model tokenizer, and word-level BIO labels are aligned to the resulting subword sequence. The maximum sequence length is 128 tokens, and the notebook pads each tokenized example to this fixed length before batch collation. This fixed-length policy is held constant across all 24 runs.

D. ADAPTATION STRATEGIES

Full fine-tuning: All 108.895 million task-relevant model parameters are trainable. The learning rate is 2×10^{-5} with cosine decay.

LoRA: The pretrained backbone is frozen while low-rank updates are introduced through PEFT target-module suffixes `query`, `key`, `value`, and `dense`. Thus, the reported configuration should be reproduced by using these exact module-name matches rather than by interpreting "dense" as a single attention projection. The implemented configuration uses rank $r = 32$, scaling factor $\alpha = 64$, dropout 0.05, and the token-classification task type. The resulting trainable parameter count is 5.312 million, or 4.88% of the full-fine-tuning count. The learning rate is 3×10^{-4}.

QLoRA: QLoRA uses the same LoRA configuration while storing the frozen backbone with 4-bit NF4 quantization and double quantization. The classifier is excluded

TABLE 1. Representative literature and relationship to the present study.

Reference	Model / method	Task or corpus	Main emphasis	Relevance here
Li et al. [1]	BC5CDR	Chemical/disease NER + relation	Benchmark resource	Dataset basis
Devlin et al. [4]	BERT	General NLP	Bidirectional pretraining	Encoder foundation
Lee et al. [2]	BioBERT	Biomedical text mining	Domain pretraining	Biomedical baseline
Beltagy et al. [6]	SciBERT	Scientific NLP	Scientific vocabulary	Domain-model context
Gu et al. [7]	PubMedBERT	Biomedical NLP	Biomedical pretraining from scratch	Backbone used here
Sun et al. [19]	BERT/MRC	Six BioNER datasets	MRC formulation	Alternative NER formulation
Giorgi and Bader [3]	Transformer systems	Biomedical NER	Reliability	Motivation for robust evaluation
Kim and Kang [16]	Biomedical NER models	Novel entities	Generalization	Motivation for cautious claims
Houlsby et al. [10]	Adapters	NLP transfer	Parameter efficiency	PEFT lineage
Li and Liang [11]	Prefix tuning	Generation	Continuous prompts	PEFT alternative
Lester et al. [12]	Prompt tuning	NLP generation	Soft prompts	PEFT alternative
Hu et al. [14]	LoRA	PLM adaptation	Low-rank updates	Primary comparison method
Dettmers et al. [15]	QLoRA	LLM fine-tuning	4-bit NF4 + LoRA	Quantized comparison method
Chen et al. [25]	PETuning methods	NLP tasks	Fair evaluation	Experimental caution
Xu et al. [26]	PEFT variants	Generation	Sample/model-size effects	Data-size motivation
Wang et al. [27]	AdaMix	NLU/NLG	Mixture of adaptations	PEFT context
Moosavi et al. [28]	Adaptable adapters	NLP transfer	Adaptive modules	PEFT context
Üstün and Cooper Stickland [29]	PEFT	Machine translation	Conditions for PEFT success	Data-size context
Pfeiffer et al. [23]	AdapterFusion	Transfer learning	Adapter composition	Modular adaptation
Poth et al. [30]	Adapters library	NLP tasks	Reproducible adapter tooling	Implementation context
Alahmari et al. [31]	QLoRA	LLM fine-tuning	Repeatability	Two-seed motivation
Keloth et al. [21]	Instruction-tuned LLMs	BioNER	Generative BioNER	Modern comparison context
Obeidat et al. [22]	LLM/encoder comparison	BioNER	Model-family comparison	Modern comparison context

from quantization, and float16 is used for computation. The trainable parameter count remains 5.312 million. The learning rate is 3×10^{-4}. The configuration follows the central memory-saving principles of QLoRA while adapting them to encoder token classification [15].

E. OPTIMIZATION AND REPRODUCIBILITY

All configurations use cosine learning-rate decay, a 6% warmup ratio, weight decay of 0.01, gradient accumulation of two steps, an effective batch size of 32, label smoothing of 0.05, a maximum of four epochs, and early stopping with patience two based on validation F1. The full-fine-tuning learning rate is 2×10^{-5}, while the LoRA and QLoRA learning rate is 3×10^{-4}. These are method-specific training settings from the notebook and are therefore reported explicitly rather than presented as an identical learning rate across methods. A checkpoint is saved each epoch and automatic resume is enabled. Gradient checkpointing is used to reduce activation memory. The implementation uses Python 3.10, PyTorch 2.5.1 with CUDA 12.1, and a Windows 10 environment. The software stack follows established open-source implementations for transformer training and numerical computing [32]–[35]. Experiments were executed on an NVIDIA GeForce RTX 3050 laptop GPU with approximately 4.3 GB reported VRAM. Runtime measurements were recorded with the notebook setting `CUDA_LAUNCH_BLOCKING=1`. The resulting times therefore describe this implementation and should not be interpreted as hardware-independent throughput figures.

Table 2 lists the training settings for each method, and Table 3 records the environment used for the resource measurements. Keeping these details in the Methods section makes the experimental conditions explicit alongside the protocol.

The memory and timing figures belong to the particular software and hardware setup used for these runs. Driver and CUDA versions, the transformer stack, quantization implementation, and the distribution of sequence lengths can all alter the observed allocation or elapsed time. A rerun should first reproduce the documented environment as closely as possible. Small memory differences are especially sensitive to such implementation overhead, even when the model configuration itself is unchanged.

F. EVALUATION METRICS

The main score is strict entity-level F1. A predicted mention contributes a true positive only when its class and both span endpoints agree with the reference annotation. Precision and recall are calculated as

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \tag{1}$$

and

$$F1 = \frac{2PR}{P + R}. \tag{2}$$

Token accuracy is included as a supporting metric rather than the main measure because the O label is dominant and can mask entity-level mistakes. For each configuration, the two runs are summarized by their mean and standard deviation. Validation F1 controls early stopping and checkpoint choice; only after that decision is fixed is the held-out test split evaluated. The selected checkpoint is restored before the test pass, keeping test performance outside the model-selection step.

Because the task is token classification, subword labels require explicit alignment. Special tokens and padding positions are masked from the loss, and the first subword of an original word receives the word-level BIO label while continuation pieces are assigned the ignore label (-100). The resulting sequences are padded or truncated to 128 tokens. These details matter because inconsistent subword treatment

TABLE 2. Detailed training configuration for the three adaptation strategies.

Category	Full fine-tuning	LoRA	QLoRA
Backbone	PubMedBERT base	PubMedBERT base	PubMedBERT base
Trainable parameters	108.895 M	5.312 M	5.312 M
Learning rate	2×10^{-5}	3×10^{-4}	3×10^{-4}
LoRA rank	–	32	32
LoRA alpha	–	64	64
LoRA dropout	–	0.05	0.05
Target modules	–	Q/K/V/dense	Q/K/V/dense
Quantization	none	none	NF4, 4-bit
Double quantization	–	–	enabled
Compute dtype	model default/BF16 support	model default/BF16 support	FP16
Max sequence length	128	128	128
Effective batch size	32	32	32
Gradient accumulation	2	2	2
Warmup ratio	0.06	0.06	0.06
Weight decay	0.01	0.01	0.01
LR schedule	cosine	cosine	cosine
Maximum epochs	4	4	4
Early stopping	patience 2	patience 2	patience 2
Seeds	42, 123	42, 123	42, 123

TABLE 3. Execution environment used for the reported experiments.

Item	Recorded value
Operating system	Windows 10
Python	3.10
PyTorch	2.5.1 + CUDA 12.1
GPU	NVIDIA GeForce RTX 3050 Laptop GPU
Reported VRAM	approximately 4.3 GB
Precision support	BF16 supported
Evaluation	seqeval strict entity-level
Dataset loader	Hugging Face Parquet revision
Checkpoint policy	per epoch + automatic resume

can change both boundary behavior and the measured entity-level F1.

G. EXPERIMENTAL AUDIT AND CONTROLLED-COMPARISON RULES

The master notebook implements several safeguards that are material to interpretation of the results. First, the BC5CDR data are loaded through the Hugging Face auto-converted Parquet revision because the original dataset loading script is not compatible with the project environment; the label mapping is then explicitly restored to the five-class BIO scheme. Second, the same four training sizes are evaluated for every method, and the sampled training subsets are fixed so that the three methods see the same sentences at a given size. Third, validation F1 is the only model-selection signal: `load_best_model_at_end` is enabled, and the test split is evaluated only after the selected checkpoint has been restored.

The QLoRA implementation contains an additional boundary that is important for reproducibility. The PubMedBERT backbone is loaded with 4-bit NF4 quantization, double quantization, and float16 computation, while the newly initialized classifier is excluded from the quantization boundary. The revised implementation treats failure of 4-bit initialization as a hard error rather than silently switching to a different partial-freezing method. The notebook also uses gradient checkpointing and records the maximum CUDA memory allocation observed during the recorded run. These implementation details explain why the reported QLoRA measurement should not be interpreted as a generic memory requirement for every 4-bit biomedical NER implementation.

Finally, the notebook supports checkpoint resume and stores a permanent result record for each run. The held-out test scores were explicitly backfilled for all 24 completed runs before the headline tables were generated. This distinction is important because validation scores are model-selection scores whereas the test scores in this paper are the final held-out measurements. The complete run-level tables below retain every recorded value, including the unusually short 0.01-min runtime recorded for LoRA with seed 42 at 500 training sentences; that value is preserved for auditability but is not used to characterize the typical PEFT runtime because it is inconsistent with the corresponding seed-123 run and the other LoRA timings.

H. SYSTEM ARCHITECTURE

Fig. 1 summarizes the complete experimental pipeline. The architecture deliberately separates data preparation, backbone selection, adaptation, evaluation, and deployment so that each stage can be reproduced independently.

IV. EXPERIMENTAL RESULTS

A. FULL-DATA PERFORMANCE

Table 4 reports the mean and standard deviation across the two seeds for the complete training set. Full fine-tuning reaches a test F1 of 0.8913. LoRA reaches 0.8955, while QLoRA reaches 0.8964. The absolute differences are small on the full dataset, indicating that freezing most of the backbone does not prevent competitive adaptation in this setting. The QLoRA result is 0.0051 F1 above full fine-tuning, while using the same 5.312 million trainable parameters as LoRA.

The validation results follow the same pattern. Fig. 2 summarizes the full-data test metrics, while the validation results provide the checkpoint-selection view. LoRA obtains

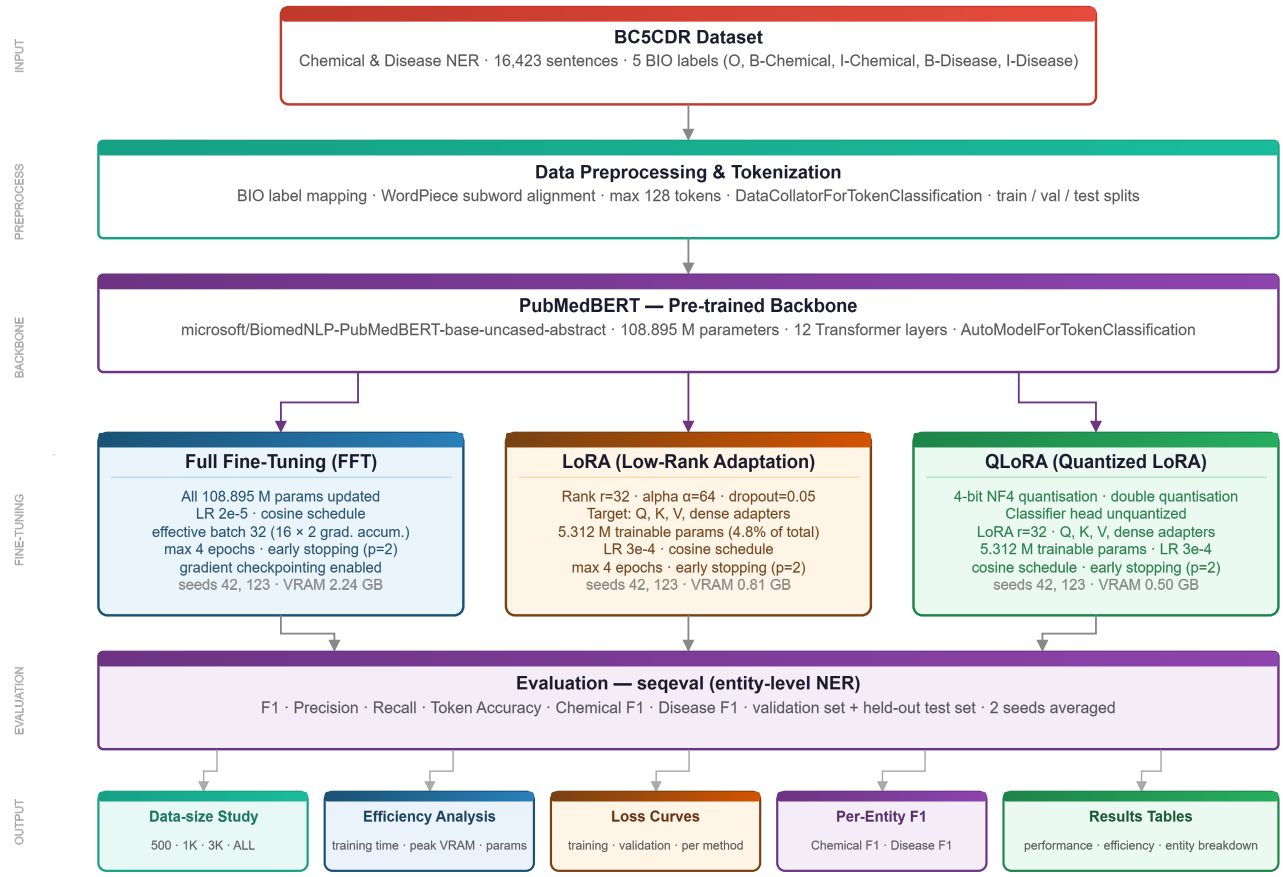


FIGURE 1. **Experimental architecture supplied with the project: BC5CDR input, BIO preprocessing and WordPiece alignment, PubMedBERT backbone, the three controlled adaptation branches, strict seqeval evaluation, and the resulting data-size, efficiency, convergence, entity-level, and tabular outputs. The diagram reports the implemented LoRA rank ($r = 32$), 4-bit NF4 QLoRA configuration, parameter counts, and measured peak CUDA allocation.**

TABLE 4. **Full-data validation and test performance (mean ± standard deviation over two seeds).**

	Validation				Test			
Method	F1	Precision	Recall	Accuracy	F1	Precision	Recall	Accuracy
Full FT	0.9012±0.0010	0.8899±0.0026	0.9130±0.0048	0.9795±0.0004	0.8913±0.0040	0.8714±0.0012	0.9120±0.0049	0.9784±0.0006
LoRA	0.9072±0.0005	0.8966±0.0007	0.9179±0.0001	0.9802±0.0002	0.8955±0.0011	0.8768±0.0007	0.9150±0.0015	0.9783±0.0003
QLoRA	0.9054±0.0014	0.8927±0.0036	0.9186±0.0011	0.9793±0.0003	0.8964±0.0013	0.8771±0.0022	0.9166±0.0022	0.9782±0.0002

validation F1 0.9072 and QLoRA 0.9054, compared with 0.9012 for full fine-tuning. Across the two full-data seeds, the test-F1 ranges are 0.8885–0.8941 for full fine-tuning, 0.8947–0.8963 for LoRA, and 0.8955–0.8973 for QLoRA. The corresponding ranges show that the PEFT differences are stable in direction in these two runs, but their absolute magnitude remains small enough that stronger statistical claims would require additional seeds. The standard deviations are small for all methods, although two seeds are not sufficient to support strong statistical significance claims. Accordingly, the differences are interpreted descriptively rather than as evidence of a universal ranking.

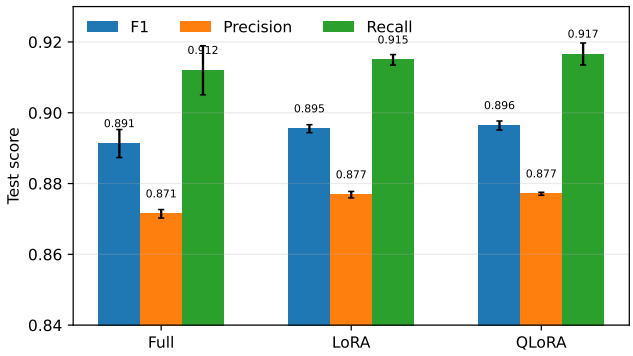


FIGURE 2. **Full-data test metrics for full fine-tuning, LoRA, and QLoRA.**

TABLE 5. Test F1 across training-set sizes.

Method	500	1,000	3,000	All
Full FT	0.7098	0.8230	0.8758	0.8913
LoRA	0.8020	0.8552	0.8856	0.8955
QLoRA	0.7853	0.8529	0.8878	0.8964

TABLE 6. Full-data computational measurements.

Metric	Full FT	LoRA	QLoRA
Trainable parameters (M)	108.895	5.312	5.312
Relative parameters	100%	4.88%	4.88%
Peak CUDA allocation (GB)	2.241	0.805	0.496
Training time (min)	9.26	16.88	21.43

B. DATA-SIZE ABLATION

The strongest separation appears when training data are limited. Table 5 shows test F1 across the four training sizes. At 500 sentences, full fine-tuning obtains 0.7098, whereas LoRA and QLoRA obtain 0.8020 and 0.7853. The LoRA–full-fine-tuning gap is therefore 0.0922 F1 at the smallest training size. At 1,000 sentences the corresponding values are 0.8230, 0.8552, and 0.8529. At 3,000 sentences, the methods become much closer, with F1 values of 0.8758, 0.8856, and 0.8878. With all training data, the values converge further to 0.8913, 0.8955, and 0.8964.

Fig. 3 provides the corresponding multi-metric view of the same data-size experiment. The data-size curve also shows a clear diminishing-gap pattern. Relative to full fine-tuning, the LoRA test-F1 gain changes from +0.0922 at 500 samples to +0.0322 at 1,000, +0.0098 at 3,000, and +0.0042 at the full training size. QLoRA follows +0.0755, +0.0299, +0.0120, and +0.0051, respectively. Thus, the largest empirical separation occurs before the full dataset is available. Table 11 summarizes the same milestones as retained full-data F1 percentages.

The data-size curve suggests that the value of PEFT is not limited to reducing memory. In this experiment, adapter methods retain more of the pretrained representation when only a small amount of task supervision is available. The observation is empirical and task-specific: it does not establish that LoRA or QLoRA will always outperform full fine-tuning in low-resource biomedical settings. It does, however, show why a full-data benchmark alone can conceal an important operational difference.

C. COMPUTATIONAL EFFICIENCY

Table 6 summarizes the full-data resource measurements. Full fine-tuning updates 108.895 million parameters, while LoRA and QLoRA update 5.312 million, corresponding to 4.88% of the full-fine-tuning trainable count. Measured peak CUDA memory allocation is 2.241 GB for full fine-tuning, 0.805 GB for LoRA, and 0.496 GB for QLoRA. Thus, relative to full fine-tuning, the measured memory reduction is approximately 64.1% for LoRA and 77.9% for QLoRA. Recorded training-loop wall-clock time is 9.26 min, 16.88 min, and 21.43 min, respectively. The timer covers the training loop and does not include dataset preparation, model construction, test evaluation, or artifact upload. The longer adapter runtimes in this implementation illustrate that fewer trainable parameters do not necessarily imply shorter wall-clock training; quantization can add computational overhead.

Fig. 4 visualizes the same full-data resource measurements. The resource measurements are particularly relevant for the target hardware. Prior studies of PEFT also emphasize that reducing trainable parameters does not automatically translate to proportional reductions in runtime or inference latency [26], [36]. QLoRA uses less than 0.5 GB of measured peak CUDA memory allocation under the reported configuration, although the total system memory requirement is higher because model loading, framework overhead, data processing, and the operating system also consume memory. The measurement should therefore be interpreted as peak CUDA allocation recorded by the experiment rather than a claim that the entire training system requires only 0.5 GB of physical memory.

D. NORMALIZED RESOURCE ANALYSIS

The measured resource values permit a more precise comparison than raw parameter counts alone. Relative to full fine-tuning, LoRA reduces peak CUDA allocation by 64.1% and QLoRA by 77.9%. The adapter methods reduce trainable parameters by 95.12%. In exchange, full-data recorded training-loop time increases from 9.26 min for full fine-tuning to 16.88 min for LoRA and 21.43 min for QLoRA. The results therefore describe a Pareto-style tradeoff rather than a single scalar notion of efficiency: QLoRA minimizes measured memory, full fine-tuning minimizes measured training time, and LoRA occupies an intermediate point while retaining nearly the same full-data F1 as QLoRA.

E. ENTITY-LEVEL ANALYSIS

The fixed-seed per-entity breakdown shows that Chemical recognition is easier than Disease recognition for all three methods. Fig. 5 visualizes the per-entity F1 comparison, while Table 7 gives the underlying precision, recall, F1, and support values. For reproducibility, this descriptive analysis uses the predeclared full-data seed 123 for every method and is not used as a method-selection criterion. For full fine-tuning, Chemical precision, recall, and F1 are 0.9245, 0.9478, and 0.9360, whereas Disease values are 0.8118, 0.8786, and 0.8439. LoRA reaches Chemical F1 0.9370 and Disease F1 0.8483. QLoRA reaches Chemical F1 0.9383 and Disease F1 0.8485. The pattern is consistent with the greater lexical variability and boundary ambiguity of disease mentions.

F. TRAINING DYNAMICS

The recorded loss curves provide a second view of convergence. Fig. 6 shows training and validation loss trajectories for the three adaptation methods. The curves should be interpreted together with validation F1 because a lower token-level loss does not necessarily translate into better exact en-

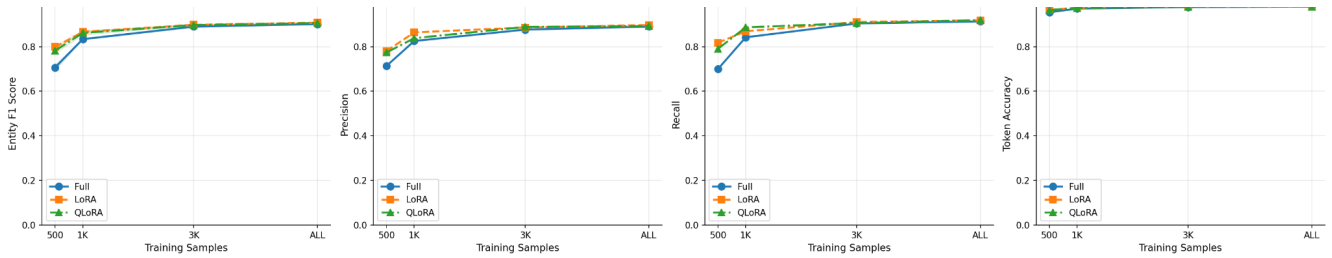


FIGURE 3. Impact of training data size on full fine-tuning, LoRA, and QLoRA across entity F1, precision, recall, and token accuracy. Values are averaged over the two recorded seeds; shaded regions indicate ± 1 standard deviation.

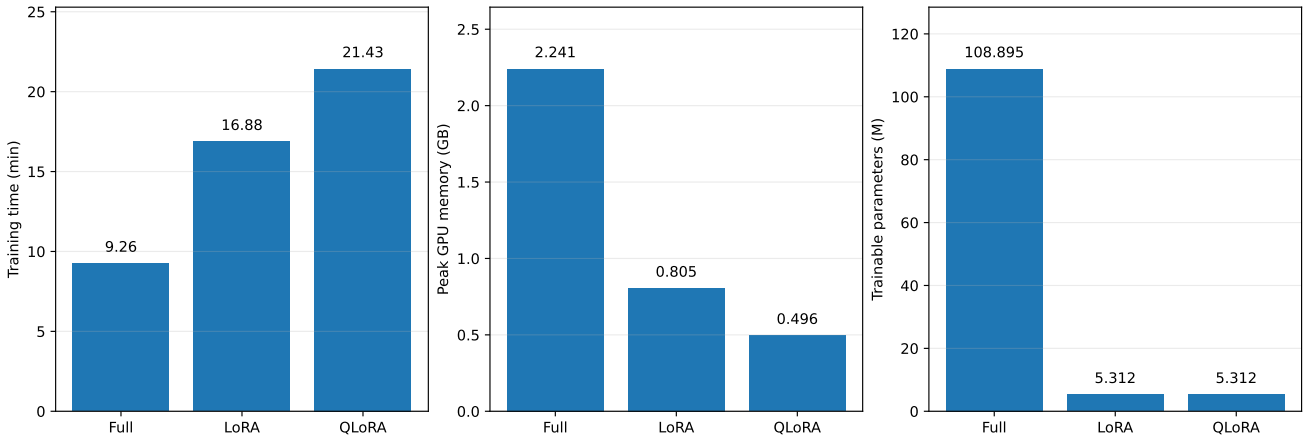


FIGURE 4. Measured full-data efficiency indicators: training time, peak CUDA memory allocation, and trainable parameters.

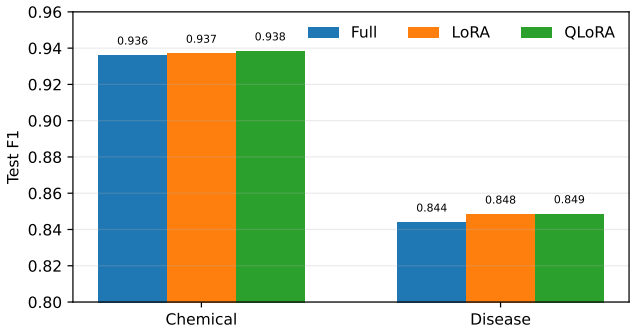


FIGURE 5. Per-entity F1 for Chemical and Disease mentions.

TABLE 7. Per-entity test performance on the full-data test set using the predeclared seed 123 for each method. This descriptive analysis is separate from the two-seed headline comparison.

Method	Entity	Precision	Recall	F1	Support
Full FT	Chemical	0.9245	0.9478	0.9360	5384
Full FT	Disease	0.8118	0.8786	0.8439	4424
LoRA	Chemical	0.9269	0.9473	0.9370	5384
LoRA	Disease	0.8206	0.8779	0.8483	4424
QLoRA	Chemical	0.9284	0.9484	0.9383	5384
QLoRA	Disease	0.8175	0.8820	0.8485	4424

tity boundaries. Early stopping limits prolonged optimization when validation F1 ceases to improve.

V. CROSS-SCALE RESULTS SYNTHESIS

The complete run record supports three quantitative observations. First, the methods become progressively closer as the amount of labeled data increases. The test-F1 gap between LoRA and full fine-tuning is 0.0922 at 500 sentences, 0.0322 at 1,000, 0.0098 at 3,000, and 0.0042 at 5,228. For QLoRA, the corresponding gaps are 0.0755, 0.0299, 0.0120, and 0.0051. The shrinking differences are visible in both the individual rows and the aggregate test grid, rather than being an artifact of a single seed.

Precision and recall move together as more labeled sentences are supplied. At 500 sentences, LoRA records 0.7796 precision and 0.8260 recall, compared with 0.7166 and 0.7032 for full fine-tuning. At 1,000 sentences, LoRA retains the recall advantage and its precision rises to 0.8462. Once all training sentences are available, the precision values fall within 0.8714–0.8771 and recall within 0.9120–0.9166. The large F1 separation at the smallest data size therefore reflects changes in both error components, not a single metric moving in isolation.

Third, memory reduction and runtime move in different directions. QLoRA lowers measured full-data peak CUDA allocation by 77.9% relative to full fine-tuning, but its mean full-data runtime is 21.43 min versus 9.26 min for full fine-tuning. LoRA sits between those extremes in memory and runtime. This result is important for deployment planning

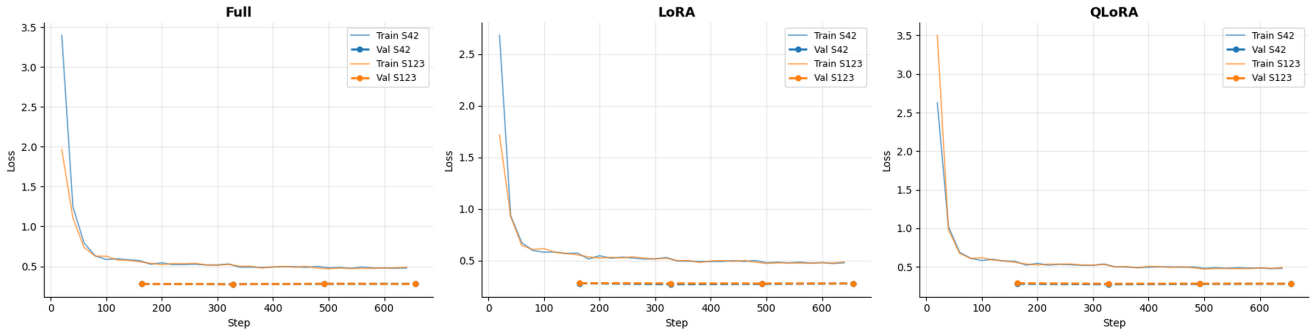


FIGURE 6. **Training and validation loss trajectories recorded during the experiments.**

because a system constrained by GPU memory can rationally accept longer training, whereas a system constrained by retraining throughput may make a different engineering choice. The experiment therefore supports reporting a resource vector rather than a single efficiency score.

The two-seed record also shows that the direction of the full-data F1 differences is stable within the tested seeds: both LoRA test scores (0.8947, 0.8963) exceed both full-fine-tuning scores (0.8885, 0.8941), and both QLoRA test scores (0.8955, 0.8973) exceed both full-fine-tuning scores. This observation is descriptive only. With two seeds, it does not establish statistical significance or generalize beyond the tested configuration.

A. TEST-SET AGGREGATE GRID ACROSS TRAINING SIZES

Table 8 reports every method–size combination as mean ± standard deviation across the two seeds for all four test metrics. This table complements the F1-only data-size curve by exposing precision, recall, and token accuracy at every operating point.

B. SEED STABILITY AND RUNTIME SCALING

The run-level record also supports two audit views that are not visible in the headline means. Table 9 gives the observed test-F1 range across the two seeds for each method and training size, while Table 10 gives the mean runtime and peak CUDA allocation for each method–size cell. Both tables summarize the same 24 runs; they do not represent additional experiments.

Among the two recorded seeds, QLoRA at 500 sentences shows the widest F1 separation, with a range of 0.0238. At 5,228 sentences the ranges shrink to 0.0056 for full fine-tuning, 0.0016 for LoRA, and 0.0018 for QLoRA. Thus, the smallest training condition shows more sensitivity to initialization in these runs. The evidence is still descriptive because two seeds are insufficient for a formal significance analysis, which is why the individual observations remain visible in the appendix.

Elapsed training time generally rises with the size of the training subset, while the measured peak allocation is comparatively stable once the subsets become larger. The 0.7125-GB LoRA mean at 500 sentences follows directly from the two recorded allocations, including the 0.620-GB run; that observation is retained without correction. The same rule applies to the 0.01-min timing entry. For the principal efficiency comparison, the manuscript uses the full-data measurements rather than averaging resource values over the four data sizes.

C. DATA-EFFICIENCY MILESTONES

A useful way to interpret the data-size experiment is to express each reduced-data score as a fraction of the same method's full-data test F1. This avoids comparing a low-data score with a different method's full-data score and directly answers how much of each method's eventual benchmark performance is retained when annotation is limited.

At 500 sentences, LoRA retains 89.56% of its own full-data F1, compared with 79.64% for full fine-tuning. QLoRA retains 87.61%. By 3,000 sentences, all three methods retain at least 98.26% of their corresponding full-data F1. This is a more precise description of the data-efficiency finding than a generic statement that PEFT is "better" with less data: the experiment shows that the relative distance to each method's own full-data operating point is smaller for the adapter methods in the tested low-data regimes.

D. PRECISION–RECALL BALANCE ACROSS THE DATA REGIME

The smallest training subset produces a different precision–recall balance from the full-data setting. At 500 sentences, full fine-tuning has precision 0.7166 and recall 0.7032; LoRA has 0.7796 and 0.8260; and QLoRA has 0.7742 and 0.7972. With 5,228 sentences, recall still exceeds precision for all three methods, but by only 0.0406, 0.0382, and 0.0395, respectively. The F1 separation at low data therefore reflects simultaneous changes in precision and recall rather than a shift in just one component.

The interpretation is kept at the aggregate level because the notebook does not contain a manually adjudicated taxonomy for boundary errors, nested entities, abbreviations, or semantic near-misses. For that reason, the paper does not attach a specific cause to the precision–recall changes. The reported values instead indicate where a later error analysis would be most useful. This restraint is intentional: the aggregate

TABLE 8. Held-out test results for the complete training-size grid (mean ± standard deviation for seeds 42 and 123).

Method	N	Test F1	Test Precision	Test Recall	Test Accuracy
Full	500	0.7098 ± 0.0057	0.7166 ± 0.0080	0.7032 ± 0.0033	0.9554 ± 0.0011
Full	1,000	0.8230 ± 0.0015	0.8074 ± 0.0005	0.8392 ± 0.0037	0.9703 ± 0.0001
Full	3,000	0.8758 ± 0.0022	0.8532 ± 0.0052	0.8998 ± 0.0011	0.9763 ± 0.0004
Full	5,228	0.8913 ± 0.0040	0.8714 ± 0.0012	0.9120 ± 0.0049	0.9784 ± 0.0006
LoRA	500	0.8020 ± 0.0064	0.7796 ± 0.0081	0.8260 ± 0.0046	0.9662 ± 0.0001
LoRA	1,000	0.8552 ± 0.0055	0.8462 ± 0.0071	0.8646 ± 0.0187	0.9739 ± 0.0001
LoRA	3,000	0.8856 ± 0.0016	0.8683 ± 0.0025	0.9034 ± 0.0062	0.9776 ± 0.0002
LoRA	5,228	0.8955 ± 0.0011	0.8768 ± 0.0007	0.9150 ± 0.0015	0.9783 ± 0.0003
QLoRA	500	0.7853 ± 0.0168	0.7742 ± 0.0340	0.7972 ± 0.0014	0.9654 ± 0.0023
QLoRA	1,000	0.8529 ± 0.0051	0.8244 ± 0.0067	0.8834 ± 0.0032	0.9730 ± 0.0006
QLoRA	3,000	0.8878 ± 0.0038	0.8742 ± 0.0023	0.9018 ± 0.0054	0.9781 ± 0.0000
QLoRA	5,228	0.8964 ± 0.0013	0.8771 ± 0.0022	0.9166 ± 0.0022	0.9782 ± 0.0002

TABLE 9. Seed-to-seed variation in held-out test F1 across training sizes; range is the difference between the two recorded runs.

Method	N	Seed 42	Seed 123	Range
Full	500	0.7058	0.7138	0.0080
LoRA	500	0.7975	0.8066	0.0091
QLoRA	500	0.7734	0.7972	0.0238
Full	1,000	0.8219	0.8240	0.0021
LoRA	1,000	0.8513	0.8591	0.0078
QLoRA	1,000	0.8493	0.8565	0.0072
Full	3,000	0.8774	0.8743	0.0031
LoRA	3,000	0.8867	0.8844	0.0023
QLoRA	3,000	0.8905	0.8851	0.0054
Full	5,228	0.8885	0.8941	0.0056
LoRA	5,228	0.8947	0.8963	0.0016
QLoRA	5,228	0.8955	0.8973	0.0018

TABLE 10. Mean runtime and mean peak CUDA allocation for each method and training size, computed from the two recorded seeds.

Method	N	Time (min)	Peak CUDA (GB)	Test F1
Full	500	2.56	2.2405	0.7098
LoRA	500	2.18	0.7125	0.8020
QLoRA	500	4.42	0.4940	0.7853
Full	1,000	3.40	2.2410	0.8230
LoRA	1,000	5.12	0.8050	0.8552
QLoRA	1,000	11.38	0.4960	0.8529
Full	3,000	5.99	2.2410	0.8758
LoRA	3,000	11.50	0.8050	0.8856
QLoRA	3,000	15.68	0.4960	0.8878
Full	5,228	9.26	2.2410	0.8913
LoRA	5,228	16.88	0.8050	0.8955
QLoRA	5,228	21.43	0.4960	0.8964

measurements can identify a pattern, but they cannot by themselves determine the exact type of boundary or semantic error responsible for it.

E. STATISTICAL INTERPRETATION OF THE CONTROLLED GRID

Two random seeds provide a limited repeatability check in this study. For each method and training size, the reported mean comes from the two held-out test values, while the standard deviation records the spread between them. Neither run is discarded in favor of the higher score. Since the experiment contains only two seeds, these summaries are descriptive rather than inferential, and small differences in F1, retention, or resource use are not treated as statistically significant.

The resource results are also reported as separate quantities. Each completed run records elapsed time and maximum CUDA allocation, and the full-data runs are used for the main comparison. These measures answer different engineering questions: trainable parameters describe the optimization and storage load, CUDA allocation describes observed GPU demand, and elapsed time describes the throughput of this particular implementation. Combining them into one weighted score would require an arbitrary choice about how those constraints should be valued.

The controlled grid also reduces several common sources of confounding. At a fixed training size, all methods receive the same sentences, validation set, test set, label mapping, sequence-length limit, and random-seed values. The backbone is held constant, and the test set is evaluated only after validation-based checkpoint selection. Consequently, the reported differences are interpretable as differences observed under the selected adaptation configurations rather than as differences caused by changing the dataset or backbone between methods.

VI. DISCUSSION

A. ACCURACY–EFFICIENCY TRADEOFF

The full-data results indicate that PEFT can preserve competitive NER quality while sharply reducing the number of trainable parameters. The absolute F1 differences are small, but the parameter difference is large: 108.895 million versus 5.312 million. This changes the engineering problem from updating the entire encoder to maintaining a small task-specific adapter on top of a frozen backbone. Such a design can be useful when several biomedical tasks share the same base model and only task-specific adapters must be stored.

The QLoRA result adds a second resource dimension. LoRA and QLoRA have identical trainable parameter counts in this experiment, yet QLoRA records substantially lower peak CUDA memory allocation. This is the expected direction of the quantization mechanism described by Dettmers et al. [15]. At the same time, QLoRA takes longer in the reported runs. Therefore, memory footprint and wall-clock time should not be collapsed into a single notion of efficiency.

TABLE 11. Data-efficiency milestones: test F1 at each training size and percentage of the method's full-data F1 retained.

Method	F1@500	% full	F1@1K	% full	F1@3K	% full	F1@5,228
Full	0.7098	79.64	0.8230	92.34	0.8758	98.26	0.8913
LoRA	0.8020	89.56	0.8552	95.50	0.8856	98.89	0.8955
QLoRA	0.7853	87.61	0.8529	95.15	0.8878	99.04	0.8964

B. WHY THE LOW-DATA REGIME MATTERS

The 500-sentence experiment is the clearest practical difference among the methods. Full fine-tuning starts substantially lower than both PEFT approaches. One plausible interpretation is that a large number of trainable degrees of freedom can be difficult to constrain when supervision is scarce, whereas LoRA restricts the update space. However, the present experiment does not directly measure overfitting, representation drift, or parameter-wise update norms. The interpretation should therefore remain an empirical hypothesis rather than a mechanistic proof.

At 3,000 and 5,228 sentences, the gap narrows. This suggests diminishing returns from the additional parameter restriction when sufficient labeled data are available. The result is consistent with the broader PEFT literature, in which the tradeoff between adaptation capacity and parameter efficiency depends on task, data scale, model architecture, and hyperparameter selection [14], [15], [23].

C. COMPARISON WITH PRIOR BIOMEDICAL NER WORK

Published biomedical NER studies have explored stronger architectures, alternative task formulations, and different pretrained encoders. For example, MRC-based NER avoids conventional BIO decoding and has reported strong results on BC5CDR and other biomedical datasets [19]. Other work has examined generalization to unseen biomedical concepts [16], domain-specific pretrained model families [2], [7], and instruction-based generation with large language models [21], [22]. These studies answer different questions from the present controlled comparison. The objective here is not to claim a new state of the art across all possible NER architectures, but to quantify the consequences of changing only the fine-tuning regime under a fixed encoder and benchmark. More broadly, empirical PEFT studies have shown that the apparent advantage of parameter-efficient methods depends on sample size, model size, and evaluation discipline [25]–[29], [37], [38]. This literature is directly relevant to the present data-size design because a single full-data comparison can hide a crossover between adaptation regimes. Recent work also explores dynamic-rank LoRA, unified adapter libraries, sparse adapters, and latency-aware PEFT [30], [36], [39]–[41]. These methods reinforce the point that parameter count, memory, and wall-clock cost are separate experimental dimensions.

D. REPRODUCIBILITY AS AN EXPERIMENTAL VARIABLE

The reproducibility record follows the same sequence used to obtain the results. Seeds 42 and 123 were applied to every configuration, and a given training size used the same sampled sentences for all three methods. Validation data alone determined checkpoint selection; the test split was opened only for the final measurement. The notebook, figures, result files, model artifacts, and demonstration preserve different parts of this workflow. They do not guarantee bit-for-bit agreement on another machine, but they provide enough recorded detail to trace how the reported measurements were produced.

E. RESEARCH-QUESTION SYNTHESIS

The two research questions can be answered directly from the controlled grid. First, under the full-data condition, LoRA and QLoRA remain within a narrow F1 band of full fine-tuning while using 4.88% as many trainable parameters and substantially less measured peak CUDA memory allocation. Second, the relative behavior changes most strongly at 500 training sentences, where the mean test-F1 gains over full fine-tuning are 0.0922 for LoRA and 0.0755 for QLoRA; by 5,228 sentences, the corresponding gaps are 0.0042 and 0.0051. These statements are descriptive of the present BC5CDR/PubMedBERT configuration and should not be generalized beyond the tested setting without additional seeds and datasets.

F. IMPLICATIONS FOR BENCHMARK REPORTING

The experimental design suggests several reporting practices that are useful when comparing parameter-efficient adaptation methods for biomedical NER. A single full-data F1 value is informative, but it does not expose how a method behaves when annotation is scarce or how much optimization state must be maintained. In the present study, the full-data F1 values differ by only a few thousandths, whereas the 500-sentence condition produces a much wider separation. This contrast demonstrates why a benchmark can give a different practical impression depending on whether it reports only the final full-data point or also examines the learning curve with respect to labeled data.

A parameter count alone does not describe the complete training cost. In these runs, LoRA and QLoRA update the same 5.312 million parameters, but their maximum CUDA allocations differ. Full fine-tuning, meanwhile, has the shortest full-data runtime despite updating 108.895 million parameters. The three measurements therefore capture different

constraints, and the manuscript leaves them separate rather than imposing a single composite efficiency score.

The individual seed records add context that a mean cannot provide by itself. Only two seeds are available, so the paper makes no significance claim, but the run-level values still show where initialization matters more. The largest spread occurs for QLoRA with 500 sentences, making that condition a natural candidate for further repetitions. Keeping the original records also means that additional seeds can be appended to the existing grid instead of reconstructing the experiment from summary values.

Overall F1 does not show whether the remaining errors are concentrated in one entity class. In the fixed-seed analysis, Chemical and Disease recognition remain separated even though the full-data aggregate scores are close. Listing precision, recall, F1, and support for each class therefore gives a more informative view of the benchmark, especially because the two entity types do not present the same lexical and boundary challenges.

The numerical results show patterns, but they do not by themselves establish why those patterns occur. The low-data advantage of the adapter methods could be related to their smaller update space, yet parameter drift, update norms, and overfitting trajectories were not measured here. Likewise, the lower QLoRA memory reading is compatible with the 4-bit frozen backbone, but it should not be assumed to produce the same saving on every software or hardware stack. The discussion therefore treats these explanations as hypotheses for follow-up experiments rather than as demonstrated mechanisms.

VII. ERROR ANALYSIS AND PRACTICAL IMPLICATIONS

The class-level results point to a consistent difference between Chemical and Disease recognition. Chemical mentions often have recognizable naming patterns, while disease mentions may be embedded in longer phrases and depend more strongly on context. Because Disease F1 is lower for every method, the overall score does not fully expose this weakness. A follow-up analysis should separate effects such as span length, abbreviations, synonyms, and partial boundary matches.

The measurements describe three different practical choices. Full fine-tuning is the conventional baseline and completes the recorded full-data training in the least time. LoRA cuts the trainable parameter count and measured memory demand while staying close to the full-data F1. QLoRA pushes memory use lower still, but requires more wall-clock time. The preferred option therefore depends on which limitation matters most in a deployment: GPU capacity, retraining speed, storage, or performance with little labeled data.

VIII. DEPLOYMENT AND REPRODUCIBILITY ARTIFACTS

A Gradio-based demonstration is deployed through Hugging Face Spaces. The application accepts biomedical text and provides chemical and disease entity highlighting. A model repository is also maintained for the trained artifacts. These deployment components are not part of the numerical evaluation itself; they are included to demonstrate that the research pipeline can be taken from offline training to an interactive inference workflow.

The complete workflow can be summarized as: dataset acquisition, BIO alignment, deterministic data-size sampling, PubMedBERT tokenization, one of three adaptation strategies, validation-based early stopping, held-out test evaluation, two-seed aggregation, resource measurement, and deployment. This separation makes it possible to reproduce individual stages without relying on the web application.

A. ARTIFACT AND DEPLOYMENT SPECIFICATION

The research artifacts are separated by function so that a reviewer or reproducer can distinguish source code, evidence, trained models, and presentation assets. Table 12 records the principal project endpoints used in this study.

The model repository stores the inference checkpoints produced for the three adaptation branches. Those files are treated as deployment artifacts and are not fed back into the benchmark computation. The reported full-data figures come from the two recorded test runs for each method, independently of the checkpoints exposed for interactive use.

- **Source repository:** GitHub project repository
- **Model repository:** Hugging Face model repository
- **Interactive demo:** Hugging Face Spaces demo

IX. CONTROLLED-COMPARISON VALIDITY

The comparison is controlled by fixing the parts of the experiment that are common to all methods and allowing the adaptation-specific settings to differ. Table 13 makes that division explicit. The same backbone, data partitions, subset sizes, seeds, label space, sequence limit, evaluation procedure, and checkpoint rule are used throughout. The adaptation mechanism, number of trainable weights, learning rate, and quantization state change because these define the treatments being compared.

The controls are particularly important when interpreting the smallest training sets. Giving different methods different sentences could make a performance gap reflect the sampled examples rather than the adaptation method. Similar ambiguity would arise if sequence length, test data, or checkpoint selection differed between methods. The protocol does not eliminate every source of uncertainty, but it fixes these comparison conditions so that the measured differences have a defined scope.

The comparison covers three specified PubMedBERT setups, rather than the full space of possible PEFT choices. The reported LoRA rank, target modules, learning rates, and quantization settings are fixed to the experiment being evaluated. If a later run changes the rank or optimizer, it should be recorded as a new configuration instead of being folded into the existing results. Stating this boundary explicitly helps distinguish a true reproduction from an experiment that only uses the same general method names.

TABLE 12. Principal artifacts used for reproducibility and deployment in the study.

Artifact	Function	Access / verification role
Master Jupyter notebook	Preprocessing, 24-run training grid, held-out evaluation, figures, and deployment code	Primary experimental source; numerical values in this manuscript are traced to its outputs.
Project README	Human-readable experimental description and result summary	Cross-check for configuration, research questions, and run inventory.
GitHub repository	Source, figures, documentation, and project history	Independent inspection of implementation and artifact organization.
Hugging Face model repository	Full, LoRA, and QLoRA trained model artifacts	Reusable inference checkpoints for the three adaptation methods; deployment artifacts are separate from the benchmark aggregation.
Hugging Face Spaces demo	Interactive biomedical NER interface	Demonstrates end-to-end inference; not used to compute benchmark scores.
Architecture SVG/PDF	Exact project architecture diagram	Presentation-level representation of the implemented pipeline and configurations.

TABLE 13. Controlled-comparison dimensions and their roles in the experiment.

Dimension	Treatment	Reason for control or variation
PubMedBERT backbone	Fixed	Prevents backbone choice from becoming a confounding factor.
BC5CDR partitions	Fixed	All methods use the same train, validation, and held-out test sentences.
Training-set sizes	Fixed grid	500, 1,000, 3,000, and 5,228 sentences expose data-scale behavior.
Random seeds	Fixed grid	Seeds 42 and 123 are repeated for every method and size.
Label space and tokenization	Fixed	The five-class BIO task and subword-label alignment are shared.
Checkpoint selection	Fixed	Validation F1 with patience-two early stopping is used before test evaluation.
Adaptation mechanism	Varied	Full fine-tuning, LoRA, and QLoRA are the experimental treatments.
Trainable parameter set	Varied	The methods optimize 108.895 M versus 5.312 M parameters.
Quantization	Varied	QLoRA alone applies 4-bit NF4 quantization to the frozen backbone.
Resource measurement	Same hardware	Runtime and peak CUDA allocation are measured on the same RTX 3050 environment.

The efficiency numbers describe this recorded execution environment; they are not hardware specifications. Peak CUDA allocation refers to the maximum GPU allocation observed during training, runtime refers to elapsed training time, and trainable parameters refer to the weights updated by optimization. Keeping these measures separate lets another researcher compare the constraint relevant to their own system without pretending that the three can be reduced to one universal efficiency value.

X. REPRODUCIBILITY AND DEPLOYMENT CONSIDERATIONS

The stored artifacts serve two different purposes. Reconstructing the benchmark requires the data partitions, tokenizer and label handling, adaptation settings, seeds, checkpoint rule, and evaluation implementation. Using the trained system requires only the resulting checkpoint, inference code, and interface. The two paths are deliberately separated so that deployment material cannot alter the benchmark, while a reviewer can still inspect the application without treating its interactive examples as additional test observations.

The master notebook is the primary reproduction source because it contains the data preparation, model setup, training loop, evaluation, figures, and deployment code. The saved 24-run record offers a second check and permits the summary tables to be rebuilt from recorded values rather than from plotted graphics. The README and architecture diagram provide a readable description of the same process, while the model repository and demo verify that the trained checkpoints can be used independently of the notebook. Taken together, the artifacts provide several routes for checking the work.

Several implementation details are recorded because small changes here can alter a rerun. The BC5CDR data enter through the project's Parquet representation, the five BIO classes are restored explicitly, and each method receives the same sampled sentences at a given training size. In QLoRA, the classifier is outside the quantized portion and the frozen PubMedBERT backbone uses 4-bit NF4 with double quantization. Gradient checkpointing, validation-based stopping, and checkpoint resume are part of the same procedure. Changing any of these settings may change memory demand, convergence, or the checkpoint that is ultimately evaluated.

The recorded efficiency values are tied to the execution setup. For the full-data runs, maximum CUDA allocations of 2.241, 0.805, and 0.496 GB were observed on the NVIDIA GeForce RTX 3050 laptop GPU; these values do not represent total system RAM. The associated training times, 9.26, 16.88, and 21.43 min, came from Windows 10 with Python 3.10, PyTorch 2.5.1, and CUDA 12.1. Another machine or software stack may produce different timings even if F1 remains similar. A reproduction should therefore preserve the experimental procedure and record its own environment alongside the new measurements.

The web application serves as an inference demonstration, not as another test set. It accepts biomedical text and uses the trained checkpoints to mark Chemical and Disease men-

tions. Interactive inputs remain outside BC5CDR and do not contribute to the reported accuracy values. Keeping those examples separate allows the interface to demonstrate end-to-end use without changing the held-out evaluation.

The separation is also useful for follow-up work. A researcher can preserve the benchmark controls and vary one factor, for example the adaptation settings, seed count, or biomedical dataset. A developer can instead take the trained checkpoints and interface forward without rerunning the complete 24-run grid. Documenting both uses keeps the information needed for scientific checking alongside the artifacts intended for practical reuse.

XI. PRACTICAL IMPLICATIONS

A deployment choice should consider the score together with the constraints of the target system. Using all 5,228 training sentences gives test F1 values of 0.8964 for QLoRA, 0.8955 for LoRA, and 0.8913 for full fine-tuning. These values are close, so the experiment does not justify declaring a single method best in every setting. GPU capacity, retraining frequency, and the amount of labeled data available can reasonably change the preferred configuration.

The strongest memory saving in these measurements comes from QLoRA: its peak allocation is 0.496 GB, compared with 0.805 GB for LoRA and 2.241 GB for full fine-tuning. The saving is not free in wall-clock terms. Full-data training takes 21.43 min with QLoRA, versus 16.88 min with LoRA and 9.26 min with full fine-tuning. A system with ample GPU memory and frequent retraining may therefore favor the faster options, while a system whose main constraint is GPU capacity may prefer QLoRA.

The data-size experiment provides a second decision dimension. At 500 training sentences, LoRA and QLoRA exceed full fine-tuning in mean test F1 by 0.0922 and 0.0755, respectively. As the training set grows, these differences contract to 0.0042 and 0.0051 at 5,228 sentences. This pattern suggests that adaptation strategy should be evaluated under the amount of supervision that will actually be available in a target application. Reporting only the full-data result would obscure this behavior. For researchers working with newly annotated biomedical corpora, a low-data pilot using the same fixed validation and held-out test protocol can therefore provide useful evidence before committing to a larger training run.

The results also clarify how the two parameter-efficient methods should be interpreted. LoRA and QLoRA use the same 5.312 million trainable parameters in the reported configuration, while QLoRA additionally quantizes the frozen backbone. Consequently, the comparison between them is primarily a memory-versus-training-time tradeoff within the same low-rank adaptation family, rather than a comparison of two unrelated model architectures. This distinction helps prevent the resource results from being attributed to parameter count alone.

Finally, the study suggests a reproducibility-oriented reporting practice for biomedical NER. Accuracy should be accompanied by the data-size setting, random seeds, checkpoint-selection rule, hardware environment, and resource measurements. The complete run-level record in Appendix A makes those quantities auditable, while the deployment artifacts remain separate from benchmark selection. This separation is particularly important when the same trained checkpoints are later exposed through an interactive interface, because application behavior should not be allowed to modify the held-out benchmark or retrospectively influence model selection.

XII. LIMITATIONS AND THREATS TO VALIDITY

Several limitations should be considered. First, only two random seeds are used. Mean and standard deviation therefore describe limited stochastic variation and should not be interpreted as a formal statistical test. Second, the study uses one benchmark, BC5CDR, and one biomedical encoder. Results may differ on NCBI Disease, BC2GM, JNLPBA, AnatEM, clinical notes, or other corpora. Third, the LoRA configuration is fixed rather than exhaustively tuned. A different rank, target-module set, dropout, or learning rate could change the comparison.

Fourth, the resource measurements are hardware- and software-specific. Peak CUDA allocation and runtime can change with driver versions, CUDA versions, framework versions, sequence distributions, batch scheduling, and background system load. Fifth, the study focuses on entity recognition rather than the downstream chemical–disease relation extraction task. High-quality entity recognition is necessary but not sufficient for complete biomedical knowledge extraction.

Finally, the data-size subsets are sentence-based. A sentence count is an operational proxy for annotation volume, but the number of tokens, entity mentions, and unique concepts can differ between subsets. Future experiments should control these quantities explicitly and should evaluate cross-dataset transfer and unseen-entity generalization.

The present study reports entity-level aggregate metrics but does not contain a manually adjudicated error taxonomy. To avoid inventing unsupported qualitative findings, this section defines the next error-analysis experiments rather than presenting them as completed results.

A. BOUNDARY ERRORS

Future analysis should separate exact matches from partial overlaps. A prediction such as “acute nephrotoxicity” for a gold span “nephrotoxicity” should be counted as a boundary error even if the semantic concept is related. This analysis would reveal whether Disease F1 is reduced mainly by span detection or by entity typing.

B. ENTITY-LENGTH EFFECTS

Entity span length is another factor that could be examined in a later experiment. The test mentions could be divided into short, medium, and long token spans and evaluated separately for each adaptation method. Comparing those groups with the

existing Chemical–Disease breakdown would help determine whether the lower Disease score is partly associated with longer or more structurally complex mentions.

C. NOVEL-ENTITY GENERALIZATION

Lexical repetition in a benchmark can make generalization look stronger than it is for unfamiliar entity names. A useful follow-up would hold out selected entity surface forms during training and evaluate only on forms that never appeared in the training partition. This would connect the present study with the unseen-concept generalization analysis of Kim and Kang [16]. Such a result is not reported here because the existing grid did not contain an unseen-entity split.

D. HYPERPARAMETER SENSITIVITY

The reported LoRA results correspond to one fixed setting: $r = 32$, $\alpha = 64$, dropout 0.05, with query, key, value, and dense modules targeted. This is the configuration evaluated in the present grid, not a sweep over LoRA design choices. Future runs could vary the rank and target modules to see whether the same performance pattern survives those changes. Dynamic-rank methods, including DyLoRA, offer one possible direction for that experiment [39].

E. CROSS-DATASET VALIDATION

External validation should use the same experimental controls on additional biomedical corpora, including NCBI Disease, BC2GM, and JNLPBA. The question would be whether the low-data behavior observed on BC5CDR survives when the entity categories and annotation conventions change. This step is important because the current benchmark evaluates only Chemical and Disease mentions.

F. STATISTICAL EXTENSION

Two seeds are sufficient for a reproducibility check but not for a high-powered statistical analysis. A future study should use at least five to ten seeds, report confidence intervals, and, where appropriate, apply paired statistical tests across identical examples. Bootstrapping at the entity level could also provide uncertainty intervals for F1 without relying solely on between-seed variation.

XIII. CONCLUSION

This study establishes a controlled empirical baseline for choosing how PubMedBERT should be adapted for biomedical NER when compute and labeled-data constraints are explicit. Across 24 runs on BC5CDR, full fine-tuning, LoRA, and QLoRA produce closely clustered full-data test F1 values (0.8913, 0.8955, and 0.8964), while their optimization footprints differ sharply. LoRA and QLoRA update 5.312 M parameters (4.88% of the 108.895 M full-fine-tuning count), and measured peak CUDA allocation falls from 2.241 GB to 0.805 GB and 0.496 GB, respectively.

The more consequential result appears under limited supervision. At 500 training sentences, LoRA reaches 0.8020 test F1 and QLoRA 0.7853, compared with 0.7098 for full fine-tuning. The gap contracts as training data increase, showing that a full-data benchmark can hide important differences in adaptation behavior. At the same time, QLoRA requires the longest measured training time in this implementation, so memory efficiency should not be interpreted as universal computational superiority.

The main contribution is the controlled measurement setup used to compare the three adaptation choices, not a claim that one method wins universally. The experiment fixes the backbone, partitions, subset construction, evaluation procedure, and seeds while retaining the run-level accuracy and resource records. A broader study should add other biomedical corpora, more random seeds, hyperparameter variation, and explicit tests on unseen entity names. Those additions would show whether the accuracy, memory, and data-efficiency patterns measured on BC5CDR also appear outside this benchmark. The existing grid can serve as the starting point without changing its basic comparison rules.

XIV. APPENDIX A: COMPLETE 24-RUN EXPERIMENTAL RECORD

The following tables reproduce the notebook's complete 24-run record without collapsing away seed-level measurements. Tables 14, 15, 16, and 17 provide the four training-size blocks in full. Each row contains validation F1, precision, recall, and token accuracy; held-out test F1, precision, recall, and token accuracy; wall-clock training time; peak CUDA memory allocation; and trainable parameter count. This appendix preserves the complete run-level audit trail, while the aggregate tables and figures in the main Results section provide the primary scientific interpretation before the Conclusion.

The LoRA seed-42, 500-sentence run records 0.01 min. The value is retained exactly because the purpose of these tables is complete reporting. It is treated as an audit flag rather than as evidence that LoRA normally trains in 0.01 min: the corresponding seed-123 run records 4.35 min, and all larger LoRA runs have materially longer runtimes. The full-data efficiency comparison therefore uses the two full-data runtimes directly reported by the notebook (18.16 and 15.60 min), while the manuscript does not infer a speed advantage from the anomalous 0.01-min record.

XV. APPENDIX B: VALIDATION RESULTS ACROSS THE DATA-SIZE GRID

The validation results are included separately so that the test-set analysis remains conceptually distinct from checkpoint selection. Table 18 reports the complete validation-grid summary. The values below are means over the two seeds and correspond to the same four training-set sizes used in the main experiment.

The validation curves increase monotonically with additional labeled training data for all three methods. The main distinction is the location of the low-data regime: adapter methods start from higher validation F1 at 500 examples and approach the full-data range by 3,000 examples. Because validation F1 determines checkpoint selection, these values

TABLE 14. Complete individual results for 500 training sentences. Every row is a completed notebook run; values are reported exactly as recorded.

Method	Seed	N	Validation				Test				Time	VRAM	Params
			F1	P	R	Acc.	F1	P	R	Acc.	min	GB	M
QLoRA	42	500	0.7733	0.7541	0.7935	0.9626	0.7734	0.7501	0.7982	0.9637	6.47	0.494	5.312
QLoRA	123	500	0.7897	0.7929	0.7864	0.9649	0.7972	0.7982	0.7962	0.9670	2.37	0.494	5.312
LoRA	42	500	0.7939	0.7751	0.8136	0.9648	0.7975	0.7738	0.8227	0.9663	0.01	0.620	5.312
LoRA	123	500	0.8026	0.7852	0.8208	0.9650	0.8066	0.7853	0.8292	0.9661	4.35	0.805	5.312
Full	42	500	0.6979	0.7034	0.6926	0.9528	0.7058	0.7109	0.7009	0.9546	0.99	2.240	108.895
Full	123	500	0.7149	0.7235	0.7065	0.9549	0.7138	0.7222	0.7055	0.9561	4.13	2.241	108.895

TABLE 15. Complete individual results for 1,000 training sentences. Every row is a completed notebook run; values are reported exactly as recorded.

Method	Seed	N	Validation				Test				Time	VRAM	Params
			F1	P	R	Acc.	F1	P	R	Acc.	min	GB	M
QLoRA	42	1000	0.8552	0.8295	0.8825	0.9720	0.8493	0.8196	0.8812	0.9725	12.31	0.496	5.312
QLoRA	123	1000	0.8677	0.8463	0.8901	0.9740	0.8565	0.8291	0.8857	0.9734	10.44	0.496	5.312
LoRA	42	1000	0.8615	0.8679	0.8552	0.9736	0.8513	0.8512	0.8514	0.9738	5.14	0.805	5.312
LoRA	123	1000	0.8707	0.8596	0.8822	0.9743	0.8591	0.8411	0.8779	0.9740	5.09	0.805	5.312
Full	42	1000	0.8347	0.8290	0.8404	0.9702	0.8219	0.8077	0.8366	0.9704	3.14	2.241	108.895
Full	123	1000	0.8322	0.8215	0.8431	0.9704	0.8240	0.8070	0.8418	0.9702	3.65	2.241	108.895

TABLE 16. Complete individual results for 3,000 training sentences. Every row is a completed notebook run; values are reported exactly as recorded.

Method	Seed	N	Validation				Test				Time	VRAM	Params
			F1	P	R	Acc.	F1	P	R	Acc.	min	GB	M
QLoRA	42	3000	0.8957	0.8859	0.9057	0.9780	0.8905	0.8758	0.9057	0.9781	17.42	0.496	5.312
QLoRA	123	3000	0.8979	0.8904	0.9055	0.9788	0.8851	0.8725	0.8980	0.9781	13.94	0.496	5.312
LoRA	42	3000	0.8964	0.8821	0.9111	0.9783	0.8867	0.8665	0.9078	0.9774	11.23	0.805	5.312
LoRA	123	3000	0.8993	0.8897	0.9090	0.9789	0.8844	0.8701	0.8991	0.9777	11.78	0.805	5.312
Full	42	3000	0.8923	0.8822	0.9026	0.9777	0.8774	0.8569	0.8990	0.9766	6.02	2.241	108.895
Full	123	3000	0.8873	0.8704	0.9048	0.9772	0.8743	0.8496	0.9006	0.9760	5.96	2.241	108.895

TABLE 17. Complete individual results for 5,228 training sentences. Every row is a completed notebook run; values are reported exactly as recorded.

Method	Seed	N	Validation				Test				Time	VRAM	Params
			F1	P	R	Acc.	F1	P	R	Acc.	min	GB	M
QLoRA	42	5228	0.9064	0.8952	0.9178	0.9791	0.8955	0.8774	0.9144	0.9781	22.72	0.496	5.312
QLoRA	123	5228	0.9044	0.8901	0.9193	0.9795	0.8973	0.8768	0.9188	0.9784	20.13	0.496	5.312
LoRA	42	5228	0.9075	0.8971	0.9180	0.9800	0.8947	0.8762	0.9139	0.9780	18.16	0.805	5.312
LoRA	123	5228	0.9068	0.8961	0.9178	0.9803	0.8963	0.8775	0.9160	0.9786	15.60	0.805	5.312
Full	42	5228	0.9005	0.8917	0.9096	0.9792	0.8885	0.8706	0.9071	0.9779	9.23	2.241	108.895
Full	123	5228	0.9019	0.8880	0.9164	0.9798	0.8941	0.8723	0.9169	0.9790	9.29	2.241	108.895

TABLE 18. Validation-set metrics across all data sizes (mean ± standard deviation over two seeds).

Method	N	F1	Precision	Recall	Accuracy
Full	500	0.7064±0.0120	0.7134±0.0142	0.6996±0.0098	0.9538±0.0015
Full	1000	0.8334±0.0018	0.8252±0.0053	0.8418±0.0019	0.9703±0.0001
Full	3000	0.8898±0.0035	0.8763±0.0083	0.9037±0.0016	0.9774±0.0004
Full	5228	0.9012±0.0010	0.8899±0.0026	0.9130±0.0048	0.9795±0.0004
LoRA	500	0.7982±0.0062	0.7802±0.0071	0.8172±0.0051	0.9649±0.0001
LoRA	1000	0.8661±0.0065	0.8638±0.0059	0.8687±0.0191	0.9740±0.0005
LoRA	3000	0.8978±0.0021	0.8859±0.0054	0.9100±0.0015	0.9786±0.0004
LoRA	5228	0.9072±0.0005	0.8966±0.0007	0.9179±0.0001	0.9802±0.0002
QLoRA	500	0.7815±0.0116	0.7735±0.0274	0.7899±0.0050	0.9638±0.0016
QLoRA	1000	0.8614±0.0088	0.8379±0.0119	0.8863±0.0054	0.9730±0.0014
QLoRA	3000	0.8968±0.0016	0.8882±0.0032	0.9056±0.0001	0.9784±0.0006
QLoRA	5228	0.9054±0.0014	0.8927±0.0036	0.9186±0.0011	0.9793±0.0003

should not be interpreted as independent final-test evidence; the held-out test results in the main paper are the primary generalization measure.

A. VALIDATION-TO-TEST CONSISTENCY

The full-data validation F1 values exceed test F1 by 0.0099 for full fine-tuning, 0.0117 for LoRA, and 0.0090 for QLoRA. These differences are modest relative to the low-data changes in F1 and do not indicate an anomalous collapse from validation to test performance. Nevertheless, a small validation–test difference should not be described as proof of absence of overfitting; it is simply evidence that the selected checkpoints transferred reasonably to this held-out split.

XVI. APPENDIX C: REPRODUCIBILITY PROTOCOL

The following protocol is intended to make an independent reimplementation possible without relying on undocumented notebook state. The experimental grid contains exactly $3 \times 4 \times 2 = 24$ runs: three adaptation methods, four training-set sizes, and two seeds. For each cell of this grid, the same validation split and the same held-out test split are used.

A. DETERMINISTIC EXPERIMENTAL GRID

The experiment is the Cartesian product of three methods, four training-set sizes, and two seeds. For each data size, the sentence subset is fixed before method training. Thus, method m at data size n and seed s receives the same examples as every other method at (n, s). The validation and test sets remain unchanged across the grid.

$$\mathcal{R} = \mathcal{M} \times \mathcal{N} \times \mathcal{S}, \tag{3}$$

where $\mathcal{M}$ contains FFT, LoRA, and QLoRA; $\mathcal{N} = \{500, 1000, 3000, 5228\}$; and $\mathcal{S} = \{42, 123\}$. Therefore $|\mathcal{R}| = 24$.

B. TRAINING PROCEDURE

Each run follows the same high-level sequence:

1) Set the random seed and load the fixed BC5CDR split.
2) Select the predetermined training subset.
3) Tokenize each example and align BIO labels to subword tokens.
4) Load PubMedBERT and construct the selected adaptation configuration.
5) Train for at most four epochs using cosine decay, 6% warmup, weight decay 0.01, gradient accumulation two, and effective batch size 32.
6) Evaluate on validation data after each checkpoint and apply patience-two early stopping.
7) Restore the selected checkpoint and evaluate the held-out test set.
8) Record F1, precision, recall, token accuracy, runtime, peak CUDA allocation, and trainable parameters.
9) Persist the result record so that the 24-run aggregate can be regenerated without manually transcribing metrics.

C. PSEUDOCODE

$\mathcal{N} \leftarrow \{500, 1000, 3000, 5228\}$
$\mathcal{S} \leftarrow \{42, 123\}$
$\mathcal{M} \leftarrow \{FFT, LoRA, QLoRA\}$
for each $n \in \mathcal{N}$ **do**
  $D_n \leftarrow$ fixed subset of BC5CDR training sentences of size n
  for each $m \in \mathcal{M}$ **do**
    for each $s \in \mathcal{S}$ **do**
      set random seed to s
      initialize PubMedBERT token-classification model with method m
      train on D_n using validation F1 for checkpoint selection
      evaluate selected checkpoint on held-out test split
      store metrics and resource measurements
    end for
  end for
end for
compute mean and standard deviation over s for each (m, n)

TABLE 19. Principal implementation components.

Component	Role
Python 3.10	Experiment orchestration
PyTorch 2.5.1 + CUDA 12.1	Model training
Hugging Face Transformers	Tokenizer and trainer/model APIs
PEFT	LoRA adapter construction
bitsandbytes	4-bit NF4 quantization
seqeval	Strict entity-level NER metrics
NumPy / pandas	Numerical and tabular processing
scikit-learn	Supporting analysis utilities
Gradio	Interactive inference interface
PyMuPDF	PDF text extraction
Hugging Face Hub / Spaces	Model and demo deployment

D. EVALUATION DISCIPLINE

The test set is not used to choose epochs, learning rates, checkpoints, data subsets, or the fixed seed used for the descriptive entity-level analysis. It is evaluated only after the selected checkpoint is fixed from validation data. This separation is important because repeated inspection of test performance can produce optimistic estimates. The protocol also reports token accuracy only as a secondary measure, preventing the large non-entity class from becoming the main success criterion.

XVII. APPENDIX D: IMPLEMENTATION AND DEPLOYMENT DETAILS

A. SOFTWARE STACK

Table 19 records the major software components used by the project. Version-specific behavior is important because transformer trainer APIs and quantization backends evolve rapidly.

B. QUANTIZATION BOUNDARY

The classifier head is kept outside the 4-bit quantization boundary. This is an implementation detail with practical importance: the classifier is newly initialized for the downstream label space, whereas the PubMedBERT backbone is the pretrained component being frozen and quantized. Keeping the task head in normal precision avoids applying quantization assumptions intended for pretrained backbone weights to newly initialized parameters.

C. DEPLOYMENT WORKFLOW

The project also includes an interactive inference layer. A Gradio application accepts biomedical text and PDF documents, performs text extraction where necessary, runs the trained NER model, and renders chemical and disease entities. The deployment is useful as a reproducibility artifact because it demonstrates that the trained checkpoint is not only an offline evaluation object but can also be used in an end-to-end inference workflow.

The model and demonstration are hosted at the project repositories. The paper intentionally separates deployment

TABLE 20. Abbreviations and notation.

Term	Meaning
NER	Named Entity Recognition
BioNER	Biomedical Named Entity Recognition
BC5CDR	BioCreative V Chemical Disease Relation corpus
BERT	Bidirectional Encoder Representations from Transformers
PEFT	Parameter-Efficient Fine-Tuning
FFT	Full Fine-Tuning
LoRA	Low-Rank Adaptation
QLoRA	Quantized Low-Rank Adaptation
NF4	NormalFloat4 quantization format
BIO	Beginning-Inside-Outside tagging scheme
F1	Harmonic mean of precision and recall
VRAM	Video Random Access Memory
P	Precision
R	Recall
TP	True positives
FP	False positives
FN	False negatives
r	LoRA rank
α	LoRA scaling factor
N	Number of training sentences

performance from benchmark performance: a web interface does not change the numerical evaluation, and inference examples are not used as additional test data.

D. DATA AND MODEL PROVENANCE

The backbone identifier, dataset split, training sizes, seeds, and label mapping are recorded explicitly in the project documentation. The use of a fixed model identifier is important because model repositories can contain multiple revisions. The experiment therefore records the model name rather than referring generically to "PubMedBERT." The same principle applies to the dataset representation and its Parquet-based loading route.

XVIII. APPENDIX E: ABBREVIATIONS AND NOTATION

For consistency, the following abbreviations and symbols are used throughout the manuscript. Table 20 defines the notation used in the equations, tables, and discussion.

The notation in the equations is kept consistent with the experimental tables. F1 always denotes the strict entity-level measure used for the primary comparison, whereas accuracy denotes token-level accuracy. Peak CUDA allocation denotes the maximum GPU memory allocation observed by the training implementation. Trainable parameters exclude frozen backbone parameters in the LoRA and QLoRA configurations.

XIX. APPENDIX F: REPRODUCIBILITY CHECKLIST

The following checklist consolidates the information required to reconstruct the reported comparison from the project artifacts. It is included as a compact audit aid rather than as an additional experiment.

- **Dataset:** BC5CDR is loaded from the documented Hugging Face representation, with fixed train, validation, and held-out test partitions.
- **Backbone:** PubMedBERT is used for all three adaptation strategies. The exact model identifier is `microsoft/BiomedNLP-PubMedBERT-base-uncased-abstract`.
- **Training sizes:** 500, 1,000, 3,000, and 5,228 training sentences are evaluated.
- **Seeds:** 42 and 123 are used for every method and training size.
- **Adaptation:** Full fine-tuning, LoRA ($r = 32$, $\alpha = 64$, dropout 0.05), and QLoRA with 4-bit NF4 quantization and double quantization are evaluated.
- **Evaluation:** Entity-level precision, recall, F1, and token accuracy are computed with the documented sequence-labeling evaluation procedure.
- **Checkpoint selection:** Validation F1 determines the selected checkpoint; the held-out test set is evaluated only after checkpoint selection.
- **Resource reporting:** Training-loop wall-clock time, peak CUDA allocation, and trainable parameter count are retained for each run.
- **Aggregate reporting:** Main comparisons report mean $\pm$ standard deviation across the two recorded seeds; complete run-level values are retained in Appendix A.
- **Deployment separation:** The interactive demonstration is treated as an application artifact and is not used to alter benchmark scores or test-set selection.

This checklist complements the detailed protocol in Appendix C and the implementation information in Appendix D. Together, these sections distinguish the experimental design, recorded evidence, and deployment artifacts so that the reported results can be independently reconstructed without relying on undocumented decisions.

ACKNOWLEDGMENT

The authors gratefully acknowledge Vellore Institute of Technology, Chennai, for institutional support and Dr. Sujatha M for research guidance and supervision.

REFERENCES

[1] J. Li, Y. Sun, R. J. Johnson, D. Sciaky, C.-H. Wei, R. Leaman, A. P. Davis, C. J. Mattingly, T. C. Wiegers, and Z. Lu, "Biocreative v cdr task corpus: a resource for chemical disease relation extraction," Database, 2016.
[2] J. Lee, W. Yoon, S. Kim, D. Kim, S. Kim, C. H. So, and J. Kang, "Biobert: a pre-trained biomedical language representation model for biomedical text mining," Bioinformatics, pp. 1234–1240, 2020.
[3] J. M. Giorgi and G. D. Bader, "Towards reliable named entity recognition in the biomedical domain," Bioinformatics, pp. 280–286, 2020.
[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, 2019, pp. 4171–4186.
[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems, 2017.
[6] I. Beltagy, K. Lo, and A. Cohan, "Scibert: a pretrained language model for scientific text," in Proc. EMNLP-IJCNLP, 2019, pp. 3615–3620.
[7] Y. Gu, R. Tinn, H. Cheng, M. Lucas, N. Usuyama, X. Liu, T. Naumann, J. Gao, and H. Poon, "Domain-specific language model pretraining for biomedical natural language processing," ACM Transactions on Computing for Healthcare, 2021.

[8] K. S. Kalyan, A. Rajasekharan, and S. Sangeetha, "Ammu: a survey of transformer-based biomedical pretrained language models," Journal of Biomedical Informatics, p. 103982, 2022.
[9] J. Li, Q. Wei, O. Ghiasvand, M. Chen, V. Lobanov, C. Weng, and H. Xu, "A comparative study of pre-trained language models for named entity recognition in clinical trial eligibility criteria from multiple corpora," BMC Medical Informatics and Decision Making, 2022.
[10] N. Houlsby, T. Giurgica-Tiron, S. Jastrzebski, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for nlp," in Proc. ICML, 2019, pp. 2790–2799.
[11] X. L. Li and P. Liang, "Prefix-tuning: optimizing continuous prompts for generation," in Proc. ACL-IJCNLP, 2021, pp. 4582–4597.
[12] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," in Proc. EMNLP, 2021, pp. 3045–3059.
[13] E. B. Zaken, S. Ravfogel, and Y. Goldberg, "Bitfit: simple parameter-efficient fine-tuning for transformer-based masked language-models," ACL Findings, pp. 1–13, 2022.
[14] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: low-rank adaptation of large language models," in Proc. ICLR, 2022.
[15] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: efficient finetuning of quantized llms," in Advances in Neural Information Processing Systems, 2023.
[16] H. Kim and J. Kang, "How do your biomedical named entity recognition models generalize to novel entities?" IEEE Access, pp. 31 513–31 523, 2022.
[17] E. Alsentzer, J. R. Murphy, W. Boag, W.-H. Weng, D. Jin, T. Naumann, and M. McDermott, "Publicly available clinical bert embeddings," in Proc. 2nd Clinical NLP Workshop, 2019, pp. 72–78.
[18] M. Yasunaga, J. Leskovec, and P. Liang, "Linkbert: pretraining language models with document links," in Proc. ACL, 2022, pp. 8003–8016.
[19] C. Sun, Z. Yang, L. Wang, Y. Zhang, H. Lin, and J. Wang, "Biomedical named entity recognition using bert in the machine reading comprehension framework," Journal of Biomedical Informatics, p. 103799, 2021.
[20] Z. Zhang and A. L. P. Chen, "Biomedical named entity recognition with the combined feature attention and fully-shared multi-task learning," BMC Bioinformatics, p. 458, 2022.
[21] V. K. Keloth, Y. Hu, Q. Xie, X. Peng, Y. Wang, A. Zheng, M. Selek, K. Raja, C. H. Wei, Q. Jin, Z. Lu, Q. Chen, and H. Xu, "Advancing entity recognition in biomedicine via instruction tuning of large language models," Bioinformatics, p. btae163, 2024.
[22] M. S. Obeidat, M. Sultan Al Nahian, and R. Kavuluru, "Do llms surpass encoders for biomedical ner?" in Proc. IEEE Int. Conf. Healthcare Informatics, 2025, pp. 352–358.
[23] J. Pfeiffer, A. Kamath, A. Rücklé, K. Cho, and I. Gurevych, "Adapterfusion: non-destructive task composition for transfer learning," in Proc. EACL, 2020, pp. 2439–2450.
[24] J. Pfeiffer, A. Rücklé, C. Poth, A. Kamath, I. Vulić, S. Ruder, K. Cho, and I. Gurevych, "Adapterhub: a framework for adapting transformers," in Proc. EMNLP Demonstrations, 2020, pp. 46–54.
[25] G. Chen, F. Liu, Z. Meng, and S. Liang, "Revisiting parameter-efficient tuning: are we really there yet?" in Proc. EMNLP, 2022, pp. 2612–2626.
[26] P. Xu, M. Patwary, S. Prabhumoye, V. Adams, R. Prenger, W. Ping, N. Lee, M. Shoeybi, and B. Catanzaro, "Evaluating parameter efficient learning for generation," in Proc. EMNLP, 2022, pp. 4824–4833.
[27] Y. Wang, S. Agarwal, S. Mukherjee, X. Liu, J. Gao, A. H. Awadallah, and J. Gao, "Adamix: mixture-of-adaptations for parameter-efficient model tuning," in Proc. EMNLP, 2022, pp. 5744–5760.
[28] N. Moosavi, Q. Delfosse, K. Kersting, and I. Gurevych, "Adaptable adapters," in Proc. NAACL-HLT, 2022, pp. 3742–3753.
[29] A. Üstün and A. Cooper Stickland, "When does parameter-efficient transfer learning work for machine translation?" in Proc. EMNLP, 2022, pp. 5429–5441.
[30] C. Poth, H. Sterz, I. Paul, S. Purkayastha, L. Englaender, T. Imhof, I. Vulić, S. Ruder, I. Gurevych, and J. Pfeiffer, "Adapters: a unified library for parameter-efficient and modular transfer learning," in Proc. EMNLP Demonstrations, 2023, pp. 149–160.
[31] S. S. Alahmari, L. O. Hall, P. R. Mouton, and D. B. Goldgof, "Repeatability of fine-tuning large language models illustrated using qlora," IEEE Access, pp. 153 221–153 231, 2024.
[32] T. Wolf et al., "Transformers: state-of-the-art natural language processing," in Proc. EMNLP Demonstrations, 2020, pp. 38–45.
[33] A. Paszke et al., "Pytorch: an imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems, 2019.
[34] F. Pedregosa et al., "Scikit-learn: machine learning in python," Journal of Machine Learning Research, pp. 2825–2830, 2011.
[35] C. R. Harris et al., "Array programming with numpy," Nature, pp. 357–362, 2020.
[36] B. Liao, Y. Meng, and C. Monz, "Parameter-efficient fine-tuning without introducing new latency," in Proc. ACL, 2023, pp. 4242–4260.
[37] S. Ruder, J. Pfeiffer, and I. Vulić, "Modular and parameter-efficient fine-tuning for nlp models," in Proc. EMNLP: Tutorial Abstracts, 2022, pp. 23–29.
[38] M. AkbarTajari, S. Rajaee, and M. T. Pilehvar, "An empirical study on the transferability of transformer modules in parameter-efficient fine-tuning," in Proc. EMNLP, 2022, pp. 10 617–10 625.
[39] M. Valipour, M. Rezagholizadeh, I. Kobyzev, and A. Ghodsi, "Dylora: parameter-efficient tuning of pretrained models using dynamic search-free low-rank adaptation," in Proc. EACL, 2023, pp. 3274–3287.
[40] S. He, L. Ding, D. Dong, J. Zhang, and D. Tao, "Sparseadapter: an easy approach for improving the parameter-efficiency of adapters," in Findings of EMNLP, 2022, pp. 2184–2190.
[41] Z. Hu, L. Wang, Y. Lan, W. Xu, E.-P. Lim, L. Bing, X. Xu, S. Poria, and R. Lee, "Llm-adapters: an adapter family for parameter-efficient fine-tuning of large language models," in Proc. EMNLP, 2023, pp. 5254–5276.



CHIGURUPATI VENKAT SAI KIRAN is pursuing the M.Tech. degree in Computer Science and Engineering with a specialization in Artificial Intelligence and Machine Learning at Vellore Institute of Technology, Chennai, India. He received the B.Tech. degree in Electronics and Communication Engineering from Anurag Engineering College, affiliated with Jawaharlal Nehru Technological University Hyderabad, India. His research interests include biomedical natural language processing, named entity recognition, parameter-efficient fine-tuning, transformer-based models, anomaly detection, and applied machine learning. His current research focuses on efficient and reproducible adaptation of pretrained language models for biomedical NLP, with particular interest in full fine-tuning, LoRA, and QLoRA for biomedical named entity recognition. His broader project work also includes industrial visual anomaly detection and network traffic anomaly detection.



DR. SUJATHA M is an Assistant Professor Senior Grade 2 in the School of Computer Science and Engineering at Vellore Institute of Technology, Chennai, India. She has over 15 years of teaching experience and received her Ph.D. from SASTRA University. Her research interests include reinforcement learning, artificial intelligence and machine learning applications, the Internet of Things, and assistive technologies. She has authored 15 publications indexed in Scopus and has published research articles in reputed international journals and conference proceedings in these areas. Her academic and research work includes the application of intelligent computing methods to practical problems involving learning, connected systems, and assistive technologies.